Activity Planner Application - Code Flow Explanation

This is a **Vert.x-based Activity Planner application** that allows users to manage activities with optional notifications. Here's the complete code flow:

---

1. Application Startup (MainVerticle.java)

The entry point is MainVerticle, which extends AbstractVerticle from Vert.x framework:

```
MainVerticle.start()
  │
  ├─▶ Creates ActivityService instance
  ├─▶ Creates ActivityRoutes with ActivityService
  ├─▶ Creates Router for HTTP routing
  ├─▶ Mounts API routes under "/api" prefix
  ├─▶ Sets up static file serving for web frontend (from "web" folder)
  ├─▶ Creates "/health" endpoint for health checks
  ├─▶ Deploys NotificationScheduler verticle
  └─▶ Starts HTTP server on port 8888
```

---

2. API Layer (ActivityRoutes.java)

The ActivityRoutes class handles all REST API endpoints:

Method	Endpoint	Description
GET	/api/activities	Get all activities
GET	/api/activities/:id	Get activity by ID

Method	Endpoint	Description
POST	/api/activities	Create new activity
PUT	/api/activities/:id	Update existing activity
DELETE	/api/activities/:id	Delete activity

Each handler calls the corresponding method in ActivityService and returns JSON responses.

---

3. Business Logic Layer (ActivityService.java)

The ActivityService manages activity data using an in-memory ConcurrentHashMap:

- **Sample Data**: Pre-populates with 2 sample activities on startup
- **CRUD Operations**:
  - getAllActivities() - Returns all activities
  - getActivityById(id) - Returns specific activity
  - createActivity(activity) - Creates new activity with UUID
  - updateActivity(id, activity) - Updates existing activity
  - deleteActivity(id) - Deletes activity by ID
- **Notification Query**: getUpcomingActivitiesWithNotification() - Finds activities with notifications enabled that are due within 15 minutes
- **Mark Notified**: markAsNotified(id) - Marks activity as notified to prevent duplicate alerts

All operations run in **blocking executors** to ensure thread safety.

---

4. Notification System (NotificationScheduler.java)

A separate verticle that runs periodically:

NotificationScheduler.start()

```
│
└─▶ Starts periodic timer (every 60 seconds)
    │
    └─▶ checkUpcomingActivities()
        │
        ├─▶ Query activities due within 15 minutes
        ├─▶ For each activity:
        │   ├─▶ sendNotification() → Console log
        │   └─▶ markAsNotified() → Prevent duplicate
        └─▶ Handle errors
```

Key Features:

- Checks every 60 seconds for upcoming activities
- Only notifies for activities within 15-minute window
- Each activity notifies only once (notified flag)
- Currently logs notifications to console (can be extended for email/push)

---

5. Frontend (Web Client)

The web interface is served from src/main/resources/web/:

- **index.html** - Main HTML page with form and activity list
- **css/style.css** - Styling for the UI
- **js/app.js** - JavaScript client that:
  - Loads activities on page load
  - Auto-refreshes every 30 seconds
  - Handles form submission (POST to API)
  - Renders activities with sorting by datetime
  - Provides delete functionality
  - Shows notification badges for notified activities

---

6. Data Model (Activity.java)

```
public class Activity {
   private String id;           // UUID
   private String title;          // Activity name
   private String description;     // Details
   private LocalDateTime dateTime;  // Scheduled time
   private boolean notificationEnabled; // Enable alert
   private boolean notified;       // Already notified flag
}
```

Complete Request Flow Example

```
User visits http://localhost:8888
      │
      ▼
StaticHandler serves index.html
      │
      ▼
Browser loads app.js
      │
      ▼
app.js calls GET /api/activities
      │
      ▼
ActivityRoutes.getAllActivities()
      │
      ▼
ActivityService.getAllActivities()
      │
```

▼

Returns JSON list of activities

│

▼

app.js renders activities to DOM

Technology Stack

- **Vert.x** - Reactive HTTP framework
- **In-memory storage** - ConcurrentHashMap
- **REST API** - JSON over HTTP
- **Static web** - Vert.x StaticHandler
- **Periodic jobs** - Vert.x setPeriodic()

Main_verticles.java

MainVerticle.java - Explanation

MainVerticle.java is the **main entry point** of the Activity Planner Vert.x application. It extends AbstractVerticle from the Vert.x framework, which is the fundamental unit of deployment in Vert.x.

Class Structure

java

```
public class MainVerticle extends AbstractVerticle {

  private ActivityService activityService;

  private ActivityRoutes activityRoutes;

  private NotificationScheduler notificationScheduler;
```

Instance Variables:

- activityService - Handles business logic for activities (CRUD operations)

- activityRoutes - Defines REST API routes/endpoints
- notificationScheduler - Manages periodic notification checks

---

start()

Method - Application Startup

This is the main initialization method that runs when the verticle starts:

Step 1: Initialize Services

java

```
activityService = new ActivityService(vertx);

activityRoutes = new ActivityRoutes(activityService);
```

Creates instances of the service and routes, passing necessary dependencies.

Step 2: Create Router

java

```
Router router = Router.router(vertx);
```

Creates an HTTP router to handle incoming requests.

Step 3: Setup API Routes

java

```
Router apiRouter = activityRoutes.createRouter();

router.mountSubRouter("/api", apiRouter);
```

Mounts all REST API endpoints under the /api prefix (e.g., /api/activities).

Step 4: Static File Serving

java

```
router.route("/*").handler(StaticHandler.create("web")

  .setIndexPage("index.html"));
```

Serves the web frontend from the web folder. Accessing http://localhost:8888/ returns index.html.

Step 5: Health Check Endpoint

java

```
router.get("/health").handler(ctx -> {
  ctx.response()
    .putHeader("content-type", "application/json")
    .end(Json.encodePrettily(new JsonObject().put("status", "UP")));
});
```

Creates a simple /health endpoint that returns {"status":"UP"} - useful for monitoring.

Step 6: Deploy NotificationScheduler

java

```
notificationScheduler = new NotificationScheduler();
vertx.deployVerticle(notificationScheduler)
```

Deploys the NotificationScheduler verticle as a background component.

Step 7: Start HTTP Server

java

```
vertx.createHttpServer()
  .requestHandler(router)
  .listen(8888)
```

Starts HTTP server on **port 8888** and binds the router to handle requests.

---

stop() **Method - Application Shutdown**

java

```
@Override
public void stop(Promise<Void> stopPromise) {
  System.out.println("MainVerticle stopping...");
  stopPromise.complete();
}
```

Called when the application shuts down. Currently just prints a message and completes the promise.

Request Flow Summary

```
HTTP Request (port 8888)
    │
    ├─▶ /api/*     → ActivityRoutes (REST API)
    ├─▶ /*         → StaticHandler (Web UI)
    └─▶ /health    → Health check JSON response
```

Key Points

1. **Verticle** - In Vert.x, a verticle is a deployable unit of code. This class extends AbstractVerticle.
2. **vertx** - A protected field inherited from AbstractVerticle that provides access to Vert.x runtime features.
3. **Async Patterns** - Uses Promise and Future for asynchronous operations (Vert.x style).
4. **SubRouter** - API routes are mounted as a sub-router under /api path.
5. **StaticHandler** - Vert.x built-in handler for serving static web files.

Activity.java - Data Model Explanation

Activity.java is a **POJO (Plain Old Java Object)** that represents the data model for an activity in the Activity Planner application. It is located in the model package.

Class Structure

```java
public class Activity {
  private String id;
  private String title;
  private String description;
  private LocalDateTime dateTime;
```

```
  private boolean notificationEnabled;
  private boolean notified;
}
```

Fields/Attributes

Field	Type	Description
id	String	Unique identifier for the activity (UUID)
title	String	Name/title of the activity
description	String	Detailed description of the activity
dateTime	LocalDateTime	Scheduled date and time for the activity
notificationEnabled	boolean	Flag to enable/disable notification alerts
notified	boolean	Flag to track if notification has been sent

Constructors

1. **Default Constructor**

java

```
  public Activity() { }
```

- Used for JSON deserialization and framework instantiation

2. **Parameterized Constructor**

java

```
public Activity(String id, String title, String description,
        LocalDateTime dateTime, boolean notificationEnabled)
```

- Creates an activity with all fields
- Automatically sets notified = false

Getter and Setter Methods

Getter	Setter	Description
getId()	setId(String)	Unique identifier
getTitle()	setTitle(String)	Activity name
getDescription()	setDescription(String)	Activity details
getDateTime()	setDateTime(LocalDateTime)	Scheduled time
isNotificationEnabled()	setNotificationEnabled(boolean)	Notification toggle
isNotified()	setNotified(boolean)	Notification status

Key Design Points

1. **Immutability** - Fields are private with public getters/setters

2. **No Validation** - Simple POJO without validation logic (validation handled in service layer)
3. **LocalDateTime** - Uses Java 8+ date/time API for scheduling
4. **Boolean Flags** - Two flags track notification state:
    - notificationEnabled - User preference to receive alerts
    - notified - System flag to prevent duplicate notifications

---

Usage in the Application

- **ActivityService** stores Activity objects in a ConcurrentHashMap
- **ActivityRoutes** converts between Activity objects and JSON
- **NotificationScheduler** checks activities where:
    - notificationEnabled == true
    - notified == false
    - dateTime is within 15 minutes

JSON Representation

```
json
{
  "id": "abc-123-def",
  "title": "Team Meeting",
  "description": "Weekly sync",
  "dateTime": "2024-01-15T10:00:00",
  "notificationEnabled": true,
  "notified": false
}
```

ActivityRoutes.java - REST API Route Handler

ActivityRoutes.java is the **REST API layer** in the Activity Planner application. It defines all the HTTP endpoints for CRUD (Create, Read, Update, Delete) operations on activities.

Class Structure

java

```
public class ActivityRoutes {

  private final ActivityService activityService;

  private static final DateTimeFormatter ISO_FORMATTER =
DateTimeFormatter.ISO_LOCAL_DATE_TIME;
```

Dependencies:

- activityService - Business logic layer (injected via constructor)
- ISO_FORMATTER - Formats date/time in ISO 8601 format

API Endpoints

All routes are mounted under /api prefix in MainVerticle:

Method	Endpoint	Handler	Description
GET	/activities	getAllActivities	Get all activities
GET	/activities/:id	getActivityById	Get activity by ID
POST	/activities	createActivity	Create new activity
PUT	/activities/:id	updateActivity	Update existing activity

Method	Endpoint	Handler	Description
DELETE	/activities/:id	deleteActivity	Delete activity

Route Handlers

1. getAllActivities

GET /api/activities

- Calls activityService.getAllActivities()
- Returns JSON array of all activities (status 200)
- Returns 500 on error

2. getActivityById

GET /api/activities/:id

- Extracts id from path parameter
- Calls activityService.getActivityById(id)
- Returns single activity (status 200) or 404 if not found

3. createActivity

POST /api/activities

- Parses JSON body using BodyHandler
- Extracts: title, description, dateTime, notificationEnabled
- Creates new Activity object
- Calls activityService.createActivity()
- Returns created activity with status 201
- Returns 400 for invalid JSON

4. updateActivity

PUT /api/activities/:id

- Extracts id from path and body fields
- Calls activityService.updateActivity(id, activity)

- Returns updated activity (status 200) or 404 if not found

5. deleteActivity

DELETE /api/activities/:id

- Extracts id from path
- Calls activityService.deleteActivity(id)
- Returns 204 (No Content) on success
- Returns 404 if not found

JSON Conversion Methods

Method	Purpose
convertActivityToJson(Activity)	Converts single activity to JSON string
convertActivitiesToJson(List<Activity>)	Converts list of activities to JSON array
escapeJson(String)	Escapes special characters for safe JSON output

---

Key Design Points

1. **BodyHandler** - Enables parsing of JSON request bodies
2. **Async Pattern** - Uses Vert.x Future/Promise with .onSuccess() and .onFailure()
3. **DateTime Parsing** - Uses ISO_LOCAL_DATE_TIME format for date strings

4. **Error Handling** - Returns appropriate HTTP status codes (400, 404, 500)
5. **Manual JSON** - Builds JSON strings manually instead of using a library

---

Request Flow Example

```
POST /api/activities
Content-Type: application/json

{
  "title": "Meeting",
  "description": "Team sync",
  "dateTime": "2024-01-15T10:00:00",
  "notificationEnabled": true
}

        │
        ▼
ActivityRoutes.createActivity()
        │
        ├─▶ Parse JSON body
        ├─▶ Create Activity object
        │
        ▼
ActivityService.createActivity()
        │
        ▼
Returns JSON:
{
  "id": "uuid",
```

```
  "title": "Meeting",
  ...
}
```

ActivityService.java - Business Logic Layer

ActivityService.java is the **service/business logic layer** in the Activity Planner application. It handles all data operations and contains the core functionality for managing activities.

Class Structure

java

```
public class ActivityService {
  private final ConcurrentHashMap<String, Activity> activities = new ConcurrentHashMap<>();
  private final Vertx vertx;
```

Key Components:

- activities - In-memory data store using ConcurrentHashMap (thread-safe)
- vertx - Reference to Vert.x instance for async operations

Constructor

java

```
public ActivityService(Vertx vertx) {
  this.vertx = vertx;
  addSampleActivities();
}
```

- Initializes Vertx reference
- Pre-populates with 2 sample activities on startup

Sample Data (Added on Startup)

1. **Team Meeting** - Due in 2 hours, notifications enabled
2. **Code Review** - Due in 1 day, notifications disabled

CRUD Operations

Method	Returns	Description
getAllActivities()	Future<List<Activity>>	Returns all activities
getActivityById(String id)	Future<Activity>	Returns activity by ID
createActivity(Activity)	Future<Activity>	Creates new activity with UUID
updateActivity(String id, Activity)	Future<Activity>	Updates existing activity
deleteActivity(String id)	Future<Boolean>	Deletes activity by ID

Operation Details

1. getAllActivities()

- Returns all activities as a new ArrayList
- Uses executeBlocking for thread safety

2. getActivityById(String id)

- Gets activity from HashMap
- Returns 404 error if not found

3. createActivity(Activity)

- Generates UUID for new activity
- Sets notified = false by default
- Stores in HashMap

4. updateActivity(String id, Activity)

- Preserves the notified flag from existing activity
- Updates the activity in HashMap

5. deleteActivity(String id)

- Removes from HashMap
- Returns true if deleted, false if not found

Notification-Related Methods

getUpcomingActivitiesWithNotification()

Filters activities that:

1. Have notificationEnabled = true
2. Have notified = false (not yet notified)
3. Are scheduled within the next 15 minutes

java

```
List<Activity> result = activities.values().stream()
  .filter(a -> a.isNotificationEnabled() && !a.isNotified())
  .filter(a -> a.getDateTime() != null && a.getDateTime().isAfter(now))
  .filter(a -> a.getDateTime().isBefore(now.plusMinutes(15)))
  .collect(Collectors.toList());
```

markAsNotified(String id)

- Sets notified = true for an activity
- Prevents duplicate notifications

Async Pattern

All methods follow Vert.x async pattern:

```java
public Future<Activity> getActivityById(String id) {
  Promise<Activity> promise = Promise.promise();
  vertx.executeBlocking((Handler<Promise<Activity>>) future -> {
    // Blocking operation here
    future.complete(activities.get(id));
  }).onSuccess(promise::complete).onFailure(promise::fail);
  return promise.future();
}
```

Pattern:

1. Create a Promise
2. Execute blocking code in executeBlocking
3. Complete/fail the promise
4. Return a Future

Key Design Points

1. **Thread Safety** - Uses ConcurrentHashMap for concurrent access
2. **In-Memory Storage** - No database; data lost on restart
3. **Blocking Execution** - All operations run in Vert.x worker pool
4. **Sample Data** - Pre-populated for demonstration
5. **UUID Generation** - Uses UUID.randomUUID() for unique IDs

pom.xml - Maven Configuration

pom.xml is the **Maven build configuration file** for the Activity Planner application. It defines project metadata, dependencies, and build plugins.

Project Metadata

xml

```
<groupId>com.example</groupId>
<artifactId>starter</artifactId>
<version>1.0.0-SNAPSHOT</version>
```

Element	Value	Description
groupId	com.example	Package namespace
artifactId	starter	Project name
version	1.0.0-SNAPSHOT	Current version

Properties

java

```
<vertx.version>4.4.0</vertx.version>
<jackson.version>2.13.5</jackson.version>
<junit-jupiter.version>5.9.1</junit-jupiter.version>
<maven-compiler-plugin.version>3.8.1</maven-compiler-plugin.version>
<main.verticle>com.example.starter.MainVerticle</main.verticle>
<launcher.class>io.vertx.core.Launcher</launcher.class>
```

Dependencies

Core Vert.x Dependencies

Artifact	Version	Purpose
vertx-core	4.4.0	Core Vert.x framework

Artifact	Version	Purpose
vertx-web	4.4.0	Web framework (Router, handlers)
vertx-web-client	4.4.0	HTTP client for API calls

Jackson (JSON Processing)

Artifact	Version	Purpose
jackson-databind	2.13.5	JSON serialization/deserialization
jackson-datatype-jsr310	2.13.5	Java 8 date/time support

Testing Dependencies

Artifact	Version	Purpose
vertx-junit5	4.4.0	Vert.x testing support
junit-jupiter-api	5.9.1	JUnit 5 API
junit-jupiter-engine	5.9.1	JUnit 5 test runner

Build Plugins

1. Maven Compiler Plugin

xml

<configuration>

```
  <release>17</release>
</configuration>
```

- Compiles Java 17 code

2. Maven Shade Plugin

- Creates fat JAR (all dependencies bundled)
- Configures MANIFEST.MF with:
  - Main-Class: io.vertx.core.Launcher
  - Main-Verticle: com.example.starter.MainVerticle

3. Maven Surefire Plugin

- Runs JUnit tests

4. Exec Maven Plugin

- Runs the application via mvn exec:java
- Launches Vert.x Launcher with MainVerticle

How to Build & Run

```bash
# Build the project
mvn clean package

# Run the application
mvn exec:java

# Run tests
mvn test
```

Key Points

1. **Vert.x 4.4.0** - Uses latest stable Vert.x
2. **Java 17** - Requires JDK 17+

3. **Fat JAR** - Single executable with all dependencies
4. **JUnit 5** - Modern testing framework
5. **No Spring** - Pure Vert.x stack (lightweight)

TestMainVerticle.java - JUnit Test

The JUnit test file is located at:

starter/src/test/java/com/example/starter/TestMainVerticle.java

---

Test File Structure

```
java
@ExtendWith(VertxExtension.class)
public class TestMainVerticle {

  @BeforeEach
  void deploy_verticle(Vertx vertx, VertxTestContext testContext) {
    vertx.deployVerticle(new MainVerticle())
      .onComplete(testContext.succeeding(id -> testContext.completeNow()));
  }

  @Test
  void verticle_deployed(Vertx vertx, VertxTestContext testContext) throws Throwable {
    testContext.completeNow();
  }
}
```

Annotations Used

Annotation	Purpose
@ExtendWith(VertxExtension.class)	Enables Vert.x testing support
@BeforeEach	Runs before each test method
@Test	Marks method as a test

Test Methods

1. deploy_verticle() - Runs before each test
    - Deploys MainVerticle to Vert.x instance
    - Waits for deployment to complete
    - Uses VertxTestContext for async testing
2. verticle_deployed() - Simple test
    - Verifies the verticle was deployed successfully
    - Currently just completes (placeholder test)

How to Run Tests

bash

mvn test

Test Dependencies (from pom.xml)

- vertx-junit5 - Vert.x JUnit 5 integration

- junit-jupiter-api - JUnit 5 API
- junit-jupiter-engine - JUnit 5 test runner

Current Test Coverage

⚠️ **Minimal Coverage:**

- Only tests that MainVerticle can be deployed
- No API endpoint tests
- No service layer tests
- No integration tests

Note: The test file is a basic template and would need additional tests for comprehensive coverage of the REST API and service methods.

API Endpoints to Test in Postman

Method	URL	Description
GET	http://localhost:8888/health	Health check
GET	http://localhost:8888/api/activities	Get all activities
GET	http://localhost:8888/api/activities/{id}	Get activity by ID
POST	http://localhost:8888/api/activities	Create new activity
PUT	http://localhost:8888/api/activities/{id}	Update activity

Method	URL	Description
DELETE	http://localhost:8888/api/activities/{id}	Delete 1activity